

DevOps
Laboratory Manual
2025-26

PROGRAM OUTCOMES (POs)

PO's	DESCRIPTION
PO1	Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2	Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4	Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO6	The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO7	Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO8	Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO9	Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10	Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

B.E COMPUTER SCIENCE AND ENGINEERING
Choice Based Credit System (CBCS) applicable for 2022 Scheme
SEMESTER – VI

DevOps (0:0:2) 1

(Effective from the academic year 2024-25)

Course Code	BCS608C	CIE Marks	50
Teaching Hours/Week (L: T:P)	0:0:2	SEE Marks	50
Total Number of Contact Hours	15	Exam Hours	3

Course Objectives:

This course will enable students to: (List as per the requirement of your course)

1. Understand and apply the DevOps tools used in SDLC.
2. Examine the docker containerization in detail.

Preamble: The DevOps Foundation course provides a comprehensive overview of understanding the DevOps competencies needed to accelerate time-to-market by improving the flow of value through the continuous delivery pipeline. Students will map the current value stream through their delivery pipeline from idea to cash, and identify practices that will eliminate bottlenecks to workflow.

Part A

1. GitHub – Create an account and fork your application code
2. To perform version control on websites or software using Git by managing repositories through a browser-based interface and synchronizing changes with push and pull commands from the command line and GUI.
3. Install Docker, explore its containerization commands, create Docker containers using various operating system images, and deploy containerized applications using Docker and Docker Hub.
4. Design, deploy, and manage a micro services architecture on your local machine using Docker and Docker-compose
5. Install Jenkins on local or cloud environment, and configure Jenkins for first use.
6. Install and configure SonarQube on your local machine and create a Jenkins CI/CD pipeline that integrates SonarQube to perform static code analysis.
7. Demonstrate the fundamentals of Maven and Gradle, and perform their installation and setup
8. Create a Maven project, understanding the POM File, dependency management and plugins.
9. Build and run a Java application with maven, migrate the same application to Gradle.

Part B

Mini Project

Application-based mini project utilizing Git, Jenkins, and Docker, focusing on CI/CD and DevOps workflows. Example applications may include a Library Management System,

Bug/Issue Tracking System, or an Online Quiz/Examination System. (The project is not limited to these examples.)

Course Outcomes: The students will be able to:

CO1:	Apply version control concepts using Git for repository management, branching, merging, and collaboration through GitHub/GitLab.
CO2:	Implement Docker containerization for software applications, including building, running, and managing containers using Docker Hub and Docker Compose.
CO3:	Develop and manage a CI/CD pipeline using Jenkins integrated with SonarQube for automated builds, testing, and code quality analysis.
CO4:	Utilize Maven for project management, including POM files, dependency handling, and build automation, and migrate projects to Gradle.

Textbooks:

1. Effective DevOps Building a Culture of Collaboration, Affinity, and Tooling at Scale, Jennifer Davis and Ryn Daniels, June 2016: First Edition Published by O'Reilly Media Inc.

References:

1. The DevOps Handbook: How to Create World-Class Agility, Reliability, & Security in Technology Organizations, by Gene Kim, Jez Humble, 2016, It Revolution Press.
2. DevOps on the Microsoft Stack, by Wouter de Kort, Apress.

Web links / e – resources:

1. https://www.youtube.com/results?search_query=github+tutorial
2. https://www.youtube.com/results?search_query=setup+and+configure+SONAR+Q
3. <https://www.youtube.com/watch?v=A0g7I4A6GN4&pp=ygUWY29udGFpbmVycyBhbmQgZG9ja2Vycw%3D%3D>

CO's and its mapping to PO's/PSO's:

CO	P O1	P O2	P O3	P O4	P O5	P O6	P O7	P O8	P O9	P O10	P O11	PS O1	PS O2
CO1: Apply version control concepts using Git for repository management, branching, merging, and collaboration through GitHub/GitLab.	3	0	0	0	0	0	0	0	0	0	0	1	0
CO 2: Implement Docker containerization for software applications, including building, running, and managing containers using Docker Hub and Docker Compose.	3	0	0	0	0	0	0	0	0	0	0	1	0
CO 3: Develop and manage a CI/CD pipeline using Jenkins integrated with SonarQube for automated builds, testing, and code quality analysis.	3	2	0	0	0	0	0	0	0	0	0	1	2
CO 4: Utilize Maven for project management, including POM files, dependency handling, and build automation, and migrate projects to Gradle.	3	3	3	3	3	1	1	3	0	3	0	1	2

CO - PO - PSO Mapping (Justification)

- CO1 is mapped to PO1 because it requires the application of core computing and engineering fundamentals, for code organization and version management. PSO1 mapping indicates applying theoretical foundations to design optimized solutions.
- CO2 maps to PO1 since it focuses implementing the software applications to develop the solution to complex engineering problems. PSO1 mapping indicates reliance on domain-specific knowledge.
- CO3 maps to PO1 (design and development skills), PO2 (Analysing problems). PSO1 reflects domain-specific expertise, PSO2 for design and development.
- CO4 maps to PO1 (apply knowledge), PO2 (Problem analysis), PO3 (design solutions), PO4 (investigations for complex problems), PO5 (Tool usage), PO6 (Engineer and the world-To analyze societal aspects to solve complex problems), PO7(ethics), PO8(Collaborative team work), PO10 (Project Management and Finance). PSO1 and PSO2 show strong domain knowledge and continuous learning.

CO1: Students will be able to use Git and GitHub/GitLab for version control.

CO2: Students will be able to containerize applications using Docker.

CO3: Students will be able to design and implement CI/CD pipelines using Jenkins for automated build, testing, and code quality analysis.

CO4: Students will be able to manage and automate software builds using Maven and Gradle.

Git:

Git is a distributed version control software system that is capable of managing versions of source code or data. It is often used to control source code by programmers who are developing software collaboratively.

Design goals of Git include speed, data integrity, and support for distributed, non-linear workflows—thousands of parallel branches running on different computers.

As with most other distributed version control systems, and unlike most client–server systems, Git maintains a local copy of the entire repository, also known as the "repo", with history and version-tracking abilities, independent of network access or a central server. A repository is stored on each computer in a standard directory with additional, hidden files to provide version control capabilities.^[13] Git provides features to synchronize changes between repositories that share history; for asynchronous collaboration, this extends to repositories on remote machines.

Docker

Docker is an OS-level virtualization (or containerization) platform, which allows applications to share the host OS kernel instead of running a separate guest OS like in traditional virtualization. This design makes Docker containers lightweight, fast, and portable, while keeping them isolated from one another.

Standardizes the runtime environment by bundling everything (app + dependencies) into containers.

- **Portability:** Runs anywhere in local machine, cloud, on-prem servers.
- **Consistency:** Same behaviour in development, testing, and production.
- **Lightweight:** No full OS per app; containers share the host kernel.
- **Scalability:** Ideal for microservices and orchestrators like Kubernetes and Docker Swarm.
- **Efficiency:** Starts in seconds, uses fewer system resources.

Components of Docker

The following are the some of the key components of Docker:

- **Docker Engine:** Docker Engine has a core part docker daemon , that handles the creation and management of containers.
- **Docker Image:** Docker Image is a read-only template that is used for creating containers, containing the application code and dependencies.

- **Docker Hub:** It is a cloud-based repository that is used for finding and sharing the container images.
- **Dockerfile:** It is a file that describes the steps to create an image quickly.
- **Docker Registry:** It is a storage distribution system for docker images, where you can store the images in both public and private modes.

Docker Compose

Docker Compose is a tool for defining and running multi-container applications.

It is the key to unlocking a streamlined and efficient development and deployment experience.

Compose simplifies the control of your entire application stack, making it easy to manage services, networks, and volumes in a single YAML configuration file.

Then, with a single command, you create and start all the services from your configuration file.

Compose works in all environments - production, staging, development, testing, as well as CI workflows. It also has commands for managing the whole lifecycle of your application:

- Start, stop, and rebuild services
- View the status of running services
- Stream the log output of running services
- Run a one-off command on a service

Jenkins

Jenkins is an open source automation server. It helps automate the parts of software development related to building, testing, and deploying, facilitating continuous integration, and continuous delivery.

Jenkins is an open-source tool that automates various stages of the software development lifecycle. It supports building, testing, and deploying applications across multiple environments, making it a critical component in CI/CD pipelines.

SonarQube is an open-source platform developed by Sonar to integrate into software development workflows, ensuring continuous code quality and code security. It provides continuous inspection of code quality to perform automatic reviews with static analysis of code to detect bugs, vulnerabilities, security hotspots, and code smells on over 35 programming languages as well as frameworks and infrastructure technologies.

Gradle

Gradle is a build automation system that is fully open source and uses the concepts you see on Apache Maven and Apache Ant.

It uses domain-specific language based on the programming language Groovy, differentiating it from Apache Maven, which uses XML for its project configuration.

It also determines the order of tasks run by using a directed acyclic graph.

Developers use Maven for automating project builds using Java.

Maven

Maven assists in outlining how to build a particular software and its different dependencies.

It leverages an XML file to describe the project under construction, the software's dependencies on third-party modules and parts, the build order, and the necessary plugins.

Maven has pre-defined targets for tasks such as packaging and compiling.

List of Programs

1. GitHub – Create an account and fork your application code

Go to <https://github.com>

Click **Sign up**.

Enter:

- a. Email address
- b. Password
- c. Username

Verify your email address.

Complete the setup process.

Your GitHub account is now ready.

2. To perform version control on websites or software using Git by managing repositories through a browser-based interface and synchronizing changes with push and pull commands from the command line and GUI.

1. Version Control Using Git in Browser

Objective: Learn to use GitHub/GitLab/Bitbucket to manage projects.

Lab Steps:

- Sign Up at GitHub or GitLab.
- Create a New Repository:
- Click New Repository.

- Provide a name and description.
- Add a README.md file.
- Upload Files Manually:
- Click Add File > Upload File.
- Select files and commit changes.
- Modify Files Online:
- Open a file and click Edit.
- Make changes and commit.
- Check File History:
- Open a file and click History.

Expected Output:

- A version-controlled project with a history of changes.
- Version Control Using Git with Push and Pull Commands
- Objective: Use Git for version control from the terminal.

Lab Steps:

- Install Git (if not installed):
- `sudo apt update`
- `sudo apt install git -y`
- Configure Git:
- `git config --global user.name "Your Name"`
- `git config --global user.email "you@example.com"`
- Clone an Existing Repository:
- `git clone <repository_url>`
- `cd <repo_name>`
- Modify a File & Commit Changes:
- `echo "New changes" >> file.txt`
- `git add .`
- `git commit -m "Updated file"`
- Push to Remote Repository:
- `git push origin main`
- Pull Latest Changes:
- `git pull origin main`

3. Install Docker, explore its containerization commands, create Docker containers using various operating system images, and deploy containerized applications using Docker and Docker Hub.

Step-by-Step Solution:

Step 1: Install Docker

On Ubuntu (Linux):

`sudo apt update`

`sudo apt install docker.io`

`sudo systemctl start docker`

`sudo systemctl enable docker`

On Windows/macOS:

Download and install Docker Desktop from the official Docker website.

Follow the installation prompts and ensure Docker Engine is running.

Step 2: Verify Docker Installation

Step 3: Explore Basic Docker Commands

o Pull an image:

`docker pull ubuntu`

Downloads the Ubuntu operating system image from Docker Hub.

o Run a container from the image:

`docker run -it ubuntu`

-it allows interactive terminal access.

You're now inside a running Ubuntu container.

o List running containers:

`docker ps`

Shows containers that are currently active.

o List all containers (including stopped):

`docker ps -a`

o Stop a container:

`docker stop <container_id>`

Remove a container:

docker rm <container_id>

o List Docker images:

docker images

o Remove a Docker image:

docker rmi <image_name>

Step 4: Create a Sample Application (Flask Web App)

We'll containerize a basic Python web app using Flask.

Create files:

app.py

```
from flask import Flask
app = Flask(__name__)
@app.route("/")
def home():
    return "Hello from Docker!"
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

requirements.txt

flask

Dockerfile

```
# Use official Python base image
FROM python:3.8-slim
# Set working directory in container
WORKDIR /app
# Copy project files
COPY . .
# Install dependencies
RUN pip install -r requirements.txt
# Run the app
CMD ["python", "app.py"]
```

Step 5: Build Docker Image

```
docker build -t flask-app .
```

flask-app is the name of the image.

. refers to the current directory where the Dockerfile is located.

Step 6: Run the Docker Container

```
docker run -d -p 5000:5000 flask-app
```

-d runs the container in detached mode (in the background).

-p 5000:5000 maps the container port to your host system.

Visit: <http://localhost:5000>

You should see: "Hello from Docker!"

Step 7: Push the Image to Docker Hub

o Login to Docker Hub

```
docker login
```

o Tag your image

```
docker tag flask-app your_dockerhub_username/flask-app
```

o Push the image

```
docker push your_dockerhub_username/flask-app
```

4. Design, deploy, and manage a micro services architecture on your local machine using

Docker and Docker-compose

Step 1: Create project directory structure

```
mkdir microservice-app
```

```
cd microservice-app
```

```
mkdir app db
```

Step 2: Create a simple Flask-based Python app (in app/app.py)

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route('/')
```

```
def home():
```

```
    return "Hello from Microservice App!"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Step 3: Add requirements file (in app/requirements.txt)

```
flask
```

Step 4: Create Dockerfile for the app (in app/Dockerfile)

```
FROM python:3.9
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

Step 5: Add a sample database container (PostgreSQL or MySQL)

We'll simulate a simple database container using an image from Docker Hub (e.g., MySQL). No need for custom code—just configure it in docker-compose.

Step 6: Create docker-compose.yml in the root directory

```
version: '3'
services:
  web:
    build: ./app
    ports:
      - "5000:5000"
    depends_on:
      - db
  db:
    image: mysql:5.7
    restart: always
    environment:
      MYSQL_DATABASE: sampledb
      MYSQL_USER: user
      MYSQL_PASSWORD: password
      MYSQL_ROOT_PASSWORD: rootpass
    ports:
      - "3306:3306"
```

Step 7: Run the application

`docker-compose up --build`

Step 8: Test the application

- Open your browser and go to `http://localhost:5000`
- You should see: Hello from Microservice App!

Step 9: Verify Containers

`docker ps`

You will see two containers running—one for the web app and one for the db.

Step 10: Shut down the setup

`docker-compose down`

5. Install Jenkins on local or cloud environment, and configure Jenkins for first use.

Method 1: Install Jenkins on Local Computer (Windows)

Step 1: Install Java

1. Download **JDK 11 or 17** from Oracle or install OpenJDK
2. After installation, check:

`java -version`

Step 2: Download Jenkins

1. Go to the official Jenkins website.
2. Download the **Windows (.msi) installer**.
3. Run the installer → Click **Next** → **Install**.

Step 3: Open Jenkins

After installation, open browser and type:

`http://localhost:8080`

Step 4: Unlock Jenkins

1. Open this file:
`C:\Program Files\Jenkins\secrets\initialAdminPassword`
2. Copy the password.
3. Paste it into the browser.

Step 5: Install Plugins

Click **Install Suggested Plugins**.

Step 6: Create Admin User

Enter:

- Username
- Password

- Email
Click **Save and Continue**.
Jenkins is ready to use.

Method 2: Install Jenkins on Cloud (Ubuntu)

Step 1: Create Server

Use Amazon Web Services (EC2 Ubuntu).
Open **port 8080**.

Step 2: Install Java

```
sudo apt update  
sudo apt install openjdk-17-jdk -y
```

Step 3: Install Jenkins

```
sudo apt install jenkins -y
```

Step 4: Start Jenkins

```
sudo systemctl start jenkins
```

Step 5: Get Password

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Step 6: Open in Browser

```
http://your-server-ip:8080  
Paste password → Install plugins → Create admin user.
```

.....

6. Install and configure SonarQube on your local machine and create a Jenkins CI/CD pipeline that integrates SonarQube to perform static code analysis.

1. Install Java (Prerequisite)

Both SonarQube and Jenkins require Java (JDK) as a runtime environment.

Steps:

```
sudo apt update # Updates the package list  
sudo apt install openjdk-11-jdk -y # Installs OpenJDK 11  
java -version # Verifies Java installation
```

Explanation: Java is essential because SonarQube is a Java-based web application, and Jenkins is also a Java-based automation server. JDK 11 is recommended as it provides stability and compatibility for both tools.

2. Install and Configure SonarQube

SonarQube is used to analyze source code quality. It detects bugs, vulnerabilities, code smells, and duplications.

Steps:

a. Download SonarQube:

```
wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-10.2.1.78527.zip
unzip sonarqube-10.2.1.78527.zip
sudo mv sonarqube-10.2.1.78527 /opt/sonarqube
```

Explanation: Downloads and extracts SonarQube to the /opt/sonarqube directory for central management.

b. Start SonarQube:

```
cd /opt/sonarqube/bin/linux-x86-64
./sonar.sh start
```

Explanation: Starts the SonarQube service in the background.

Access it via <http://localhost:9000>.

c. Login to SonarQube:

- Open a browser → **http://localhost:9000**.
- Default credentials:

o **Username:** admin

o **Password:** admin

- Change the password after the first login.

3. Install and Configure Jenkins

Jenkins is used to **automate CI/CD processes** such as **building, testing, and deploying applications**. It integrates with SonarQube for static code analysis.

Steps:

a. Install Jenkins:

```
wget -q -O - https://pkg.jenkins.io/debian-stable/jenkins.io.key | sudo apt-key add -
sudo sh -c 'echo deb https://pkg.jenkins.io/debian-stable binary/ >
/etc/apt/sources.list.d/jenkins.list'
sudo apt update
```

```
sudo apt install jenkins -y
```

Explanation: Adds Jenkins' repository, updates the package list, and installs Jenkins.

Start Jenkins Service:

```
sudo systemctl start jenkins
```

```
sudo systemctl enable jenkins
```

Explanation: Starts Jenkins and enables it to run on system boot. Access Jenkins via **http://localhost:8080**.

c. Unlock Jenkins:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Copy the displayed password → paste it into the Jenkins setup page to unlock.

Explanation: This step unlocks Jenkins for initial setup.

Install **suggested plugins** during setup.

4. Integrate SonarQube with Jenkins

To ensure **static code analysis** is part of the **CI/CD pipeline**, integrating SonarQube with Jenkins allows **automated code quality checks** every time code is built.

Steps:

a. Install SonarQube Scanner Plugin in Jenkins:

- Jenkins Dashboard → **Manage Jenkins** → **Manage Plugins** → **Available** tab.
- Search for **SonarQube Scanner**, install it, and restart Jenkins.

b. Configure SonarQube Server in Jenkins:

- ☐ Jenkins Dashboard → **Manage Jenkins** → **Configure System**.
- ☐ Scroll to **SonarQube servers**.
- ☐ Add a new server:
 - o **Name:** My SonarQube Server
 - o **Server URL:** http://localhost:9000
 - o **Authentication Token:** Generate it in SonarQube (**My Account** → **Security** → **Generate Tokens**).
- ☐ **Explanation:** This allows Jenkins to connect securely with SonarQube for analysis.

5. Create a CI/CD Pipeline Integrating SonarQube

Two Options:

1. Freestyle Project
2. Pipeline Project (Jenkinsfile)

For Freestyle Project:

New Item → **Freestyle project** → **Source Code Management (Git)** → Enter repository URL.

Build Step → **Execute SonarQube Scanner**.

Create a **sonar-project.properties** file in the project's root:

```
sonar.projectKey=myproject
sonar.sources=src
sonar.host.url=http://localhost:9000
sonar.login=<Generated_Token>
```

For Pipeline Project (Recommended):

New Item → **Pipeline** → **Jenkinsfile**:

```
pipeline {
    agent any
    tools {
        maven 'Maven3' // Maven tool defined in Jenkins
    }
    stages {
```

```

stage('Build') {
  steps {
    sh 'mvn clean install'
  }
}
stage('SonarQube Analysis') {
  steps {
    withSonarQubeEnv('My SonarQube Server') {
      sh 'mvn sonar:sonar'
    }
  }
}
}

```

Explanation:

- **Build Stage:** Compiles and packages the application using Maven.
- **SonarQube Analysis Stage:** Runs **SonarQube scanner** integrated with Maven to analyze the source code.

6. Run the Pipeline

- Trigger the Jenkins job manually or automatically (via webhooks).
- Check the SonarQube dashboard (<http://localhost:9000>) for analysis results (bugs, code smells, vulnerabilities).

7. Demonstrate the fundamentals of Maven and Gradle, and perform their installation and setup

Maven Installation

- Install Java (`java -version`)
 - Download Maven & extract
 - Add `bin` folder to PATH
- Verify:
- `mvn -version`

Gradle Installation

- Install Java
- Download Gradle & extract
- Add `bin` to PATH
- Verify:
- `gradle -version`

8. Create a Maven project, understanding the POM File, dependency management and plugins.

Prerequisites:

Install JDK and Maven:

```
sudo dnf install java-17-openjdk-devel maven -y
```

Step 1: Create Maven Project

```
mvn archetype:generate \  
-DgroupId=com.example.mavenapp \  
-DartifactId=mavenapp \  
-DarchetypeArtifactId=maven-archetype-quickstart \  
-DinteractiveMode=false
```

Step 2: Understand pom.xml

Basic Structure:

```
<project>  
  <modelVersion>4.0.0</modelVersion>  
  <groupId>com.example.mavenapp</groupId>  
  <artifactId>mavenapp</artifactId>  
  <version>1.0-SNAPSHOT</version>  
  <dependencies>  
    <!-- Unit Testing Framework -->  
    <dependency>  
      <groupId>junit</groupId>  
      <artifactId>junit</artifactId>  
      <version>4.13.2</version>  
      <scope>test</scope>  
    </dependency>  
  </dependencies>  
  <build>  
    <plugins>  
      <!-- Compiler Plugin to set Java version -->  
      <plugin>  
        <groupId>org.apache.maven.plugins</groupId>  
        <artifactId>maven-compiler-plugin</artifactId>  
        <version>3.8.1</version>  
      </plugin>  
    </plugins>  
  </build>  
</project>
```

```
<source>1.8</source>
<target>1.8</target>
</configuration>
</plugin>
</plugins>
</build>
</project>
```

Step 3: Build the Project

```
cd mavenapp
mvn clean install
```

Step 4: Run the Java Program

If App.java has the main method:

```
mvn exec:java -Dexec.mainClass="com.example.mavenapp.App"
```

If not available, add exec-maven-plugin to pom.xml.

9 Build and Run a Java Application with Maven, Migrate the Same Application to Gradle.

A. Using Maven — already done in Lab 7

Migrate to Gradle

Step 1: Install Gradle

```
sudo dnf install gradle -y
```

Check version:

```
gradle -v
```

Step 2: Create Gradle Project

```
mkdir gradleapp && cd gradleapp
```

```
mkdir -p src/main/java/com/example/gradleapp
```

Step 3: Create App.java

File: src/main/java/com/example/gradleapp/App.java

```
package com.example.gradleapp;
```

```
public class App {
```

```
    public static void main(String[] args) {
```

```
        System.out.println("Hello from Gradle on Fedora!");
```

```
}
```

```
}
```

Step 4: Create build.gradle

```
plugins {  
    id 'java'  
    id 'application'  
}  
group = 'com.example.gradleapp'  
version = '1.0'  
sourceCompatibility = '1.8'  
repositories {  
    mavenCentral()  
}  
dependencies {  
    testImplementation 'junit:junit:4.13.2'  
}  
  
application {  
    mainClass = 'com.example.gradleapp.App'  
}
```

Step 5: Build the Gradle Project

gradle build

Step 6: Run the Application

gradle run
